

Software Modeling and Verification — Expanded

Completed Course Notes with PlantUML Diagrams and Examples

Table of contents

Preface	2
1. Overview: What is verification?	2
2. Models, Specifications, and the Verification Question	3
3. Main Verification Approaches (with diagrams)	3
3.1 Deductive Reasoning (Hoare Logic, WP, SMT)	3
3.2 Abstract Interpretation	3
3.3 Model Checking	4
3.4 Assisted Proofs (Proof Assistants)	4
4. A Small Language: Syntax and Semantics	4
4.1 Operational Semantics (big-step, informal)	4
5. Hoare Logic: Triples and Validity	5
6. Weakest Preconditions (WP) — definitions and examples	5
6.1 Example: assignment	5
6.2 Example: sequential composition	6
6.3 Example: Hoare proof with function <code>add(a,b)</code>	6
7. Loops, Invariants and Termination	7
7.1 Loop invariants (how to choose one)	7
7.2 Example: <code>sum(n)</code> using loop (iterative rewrite)	7
8. Model Checking — example and counterexample	8
9. Abstract Interpretation — example abstract domain	8

10. Combining Approaches in Practice	8
11. Additional Examples (detailed)	9
11.1 Prove <code>min(a,b)</code> correctness	9
11.2 Example: small program with a failing spec (counterexample)	10
12. Practical Tips and Tooling	10
13. Summary	11

Preface

This document expands and completes your Software Modeling and Verification notes. It contains:

- fully usable PlantUML.locks (Quarto `diagram` filter) — renderable when you run `quarto render` or build the PDF,
- additional worked examples, step-by-step proof sketches, and more detailed explanations,
- a final **Summary** section.

1. Overview: What is verification?

Software verification answers: **Does the program meet its specification?**

- **Functional properties:** what the program computes (correctness of algorithms)
- **Non-functional properties:** time, memory, security, usability

We formalize programs with mathematical **models** and express desired properties in **logic** so that tools can reason automatically or semi-automatically.

2. Models, Specifications, and the Verification Question

A program source file has *syntax* (checked by compiler) and *semantics* (meaning). Compilers generally check syntax; verification tackles semantics.

Notation: $\text{Program} \models \text{Spec}$ means the program satisfies the specification.

```
@startuml
title Overview: program, spec, correctness
[Computer-based system / programs] -left-> [Program: source + compiler]
[Specifications] -right-> [Spec: formal properties]
[Program] --> [Correct?]
[Spec] --> [Correct?]
@enduml
```

3. Main Verification Approaches (with diagrams)

3.1 Deductive Reasoning (Hoare Logic, WP, SMT)

We encode program behavior as logical formulas and discharge proof obligations with SMT solvers or proof assistants.

```
@startuml
title Deductive reasoning pipeline
[Program + Spec] --> [Generate Proof Obligations]
[Generate Proof Obligations] --> [SMT Solver / Theorem Prover]
[SMT Solver / Theorem Prover] --> [Valid / Invalid / Unknown]
@enduml
```

3.2 Abstract Interpretation

Compute an **over-approximation** of reachable states (safe but may report false alarms).

```
@startuml
title Abstract interpretation
pgm -> "Abstract Model (lattice)"
"Abstract Model (lattice)" -> "Fixpoint computation"
"Fixpoint computation" -> "Over-approximation of reachable states"
@enduml
```

3.3 Model Checking

Explicitly or symbolically enumerate reachable states and verify temporal properties (CTL/LTL). Exact for finite-state systems; faces state-explosion for larger systems.

```
@startuml
title Model checking pipeline
[System Model] --> [State Space / Kripke Structure]
[Spec (LTL/CTL)] --> [Model Checker]
[Model Checker] --> [Counterexample or Verified]
@enduml
```

3.4 Assisted Proofs (Proof Assistants)

User-guided, machine-checked proofs with tools like Coq, Isabelle — great for deep correctness theorems, less automated.

4. A Small Language: Syntax and Semantics

We consider a tiny imperative language with integers and booleans, assignments, **if** and **while**, and function calls (simple).

4.1 Operational Semantics (big-step, informal)

- Memory/state: $\mu : Var \rightarrow \mathbb{Z} \cup \{tt, ff\}$
- $Val(e, \mu)$: evaluation of expression e in state μ
- $Mem(c, \mu)$: executing command c in state μ returns new state μ' (if it terminates)

Example:

- $Mem(y := 0, \mu) = \mu[y \leftarrow 0]$

5. Hoare Logic: Triples and Validity

A Hoare triple is written:

$$\{P\} c \{Q\}$$

Meaning: if P holds in the initial state and c executes (terminates), then Q holds in the resulting state. Validity: for every state μ , $\mu \models P \Rightarrow Mem(c, \mu) \models Q$ (assuming termination).

```
@startuml
title Hoare triple idea
[P: Precondition] --> [c: Command]
[c: Command] --> [Q: Postcondition]
@enduml
```

Note: Partial correctness excludes termination reasoning. Total correctness includes termination proof (often via variants/metrics).

6. Weakest Preconditions (WP) — definitions and examples

Rules (deterministic language):

- Assignment: $WP(x := t, Q) = Q[x \leftarrow t]$
- Sequence: $WP(i_1; i_2, Q) = WP(i_1, WP(i_2, Q))$
- Conditional: $WP(\text{if } b \text{ then } i_1 \text{ else } i_2, Q) = (b \Rightarrow WP(i_1, Q)) \wedge (\neg b \Rightarrow WP(i_2, Q))$

6.1 Example: assignment

Program: $x := x + 1$ and postcondition $Q: x \geq 1$.

Compute WP:

- $WP(x := x + 1, x \geq 1) = (x + 1 \geq 1)$
- Simplify: $x \geq 0$

Thus, the Hoare triple $\{x \geq 0\} \ x := x + 1 \ \{x \geq 1\}$ is valid.

6.2 Example: sequential composition

Program:

```
y := x + 2;  
x := y - 1;
```

Postcondition: $Q: x \geq 5$.

Compute backwards:

- $WP(x := y - 1, x \geq 5) = y - 1 \geq 5 \quad y \geq 6$
- $WP(y := x + 2, y \geq 6) = x + 2 \geq 6 \quad x \geq 4$

So $\{x \geq 4\} \quad y := x + 2; \quad x := y - 1 \quad \{x \geq 5\}$

6.3 Example: Hoare proof with function add(a,b)

Consider the simple loop-based addition (from your notes):

```
add(a, b) =  
  res := a;  
  aux := b;  
  while aux > 0 do  
    res := res + 1;  
    aux := aux - 1;  
  done;  
  res
```

We want partial correctness: $\{aux = b \ \& \ res = a\}$ loop $\{res = a + b\}$ at loop exit.

Loop invariant candidate: $I: res = a + (b - aux)$ and $aux \geq 0$.

- Initialization: before loop, $res = a$ and $aux = b$ $res = a + (b - b) = a$. OK.
- Preservation: assume I and $aux > 0$ hold, after the body:
 - $res' = res + 1 = a + (b - aux) + 1 = a + (b - (aux - 1))$
 - $aux' = aux - 1$So invariant holds for next iteration.

- Termination (when loop ends): $\text{aux} = 0$ invariant gives $\text{res} = a + (b - 0) = a + b$. So postcondition holds.

Thus the loop is partially correct wrt the specification.

7. Loops, Invariants and Termination

7.1 Loop invariants (how to choose one)

- Derive invariant from desired postcondition by working backwards.
- Invariants must be strong enough to imply postcondition when combined with loop exit condition.
- For termination proofs, provide a **variant** (a well-founded integer measure) that strictly decreases each iteration and is bounded below (e.g., **aux** in the example).

7.2 Example: $\text{sum}(n)$ using loop (iterative rewrite)

Recursive $\text{sum}(n)$ in your notes can be made iterative:

```
sum_iter(n):  
  i := 0;  
  acc := 0;  
  while i <= n do  
    acc := acc + i;  
    i := i + 1;  
  done;  
  return acc;
```

Invariant: $\text{acc} = \sum_{k=0}^{i-1} k$ and $0 \leq i \leq n+1$. At termination $i = n+1$, so $\text{acc} = \sum_{k=0}^n k = n(n+1)/2$.

8. Model Checking — example and counterexample

Model checking verifies properties often expressed in temporal logics (LTL/CTL). A counterexample is a concrete trace showing a property violation.

```
@startuml
title Model checking: counterexample trace
start
:Initial state s0;
if (property holds?) then (yes)
  :Verified;
else (no)
  :Return counterexample trace (s0 -> s1 -> ... -> sk);
endif
stop
@enduml
```

Small worked example (informal): a mutual exclusion protocol with two processes. Model checker may find a trace where both enter critical section — that trace is a counterexample to mutual exclusion.

9. Abstract Interpretation — example abstract domain

Abstract interpretation uses lattices (e.g., intervals, signs, parity) to safely approximate sets of states.

Interval domain example: Track x as interval $[l, u]$. For $x := x + 1$ we map $[l, u] \rightarrow [l+1, u+1]$. This is efficient and safe but loses some precision (e.g., relational facts like $x = y + 1$).

10. Combining Approaches in Practice

- Use **static analyzers / abstract interpreters** for scalable bug-finding (e.g., taint analysis).

- Use **SMT-based verification** for verifying small but complex algorithms (e.g., sorting routines).
- Use **model checking** for concurrent/finite-state controllers and protocols.
- Use **proof assistants** to obtain end-to-end certified proofs when absolute guarantees are required.

```
@startuml
title Combined verification workflow
[Source code] --> [Front-end: parse/model]
[Front-end: parse/model] --> [Abstract interpreter (fast)]
[Front-end: parse/model] --> [SMT / Hoare proof obligations (precise)]
[Front-end: parse/model] --> [Model checker (finite-state)]
@enduml
```

11. Additional Examples (detailed)

11.1 Prove $\min(a,b)$ correctness

Program:

```
min(a, b) =
  if a < b then
    res := a
  else
    res := b
  endif;
res
```

Spec: returns $\min(a,b)$ i.e. $\text{res} \leq a \ \& \ \text{res} \leq b \ \& \ (\text{res} = a \ \text{or} \ \text{res} = b)$.

Use WP for each branch:

- If $a < b$ branch: post Q becomes $Q[\text{res} := a]$ which is $a \leq a \ \& \ a \leq b \ \& \ (a = a \ \text{or} \ a = b)$ — simplifies to $a \leq b$ needed. But since the branch is taken under $a < b$ (stronger than $a \leq b$), the triple holds.
- If not $(a < b)$ branch: symmetrical reasoning.

Combine using conditional rule and the guard conditions; the triple is valid.

11.2 Example: small program with a failing spec (counterexample)

Program:

```
x := 0;
y := 1;
if y = 0 then
  x := x + 1
else
  skip
endif;
```

Spec: final $x = 1$.

Compute WP: $\text{WP}(\text{if } y=0 \text{ then } x:=x+1 \text{ else skip, } x=1) = (y=0 \Rightarrow \text{WP}(x:=x+1, x=1)) \ \& \ (y \neq 0 \Rightarrow \text{WP}(\text{skip}, x=1))$

- $\text{WP}(x:=x+1, x=1) = x+1 = 1 \quad x = 0$.

- $\text{WP}(\text{skip}, x=1) = x = 1$.

So WP overall: $(y=0 \Rightarrow x = 0) \ \& \ (y \neq 0 \Rightarrow x = 1)$. Before the if, we had $x := 0; y := 1$ state is $x=0, y=1$. Evaluate WP: since $y \neq 0$ branch holds, we need $x=1$ but $x=0$. Hence the triple is invalid and a counterexample state exists (the actual run gives $x=0$ at the end).

12. Practical Tips and Tooling

- Express assertions and invariants early and keep them simple.
 - Use automated invariant inference when available, but validate them manually.
 - Combine quick scalable analyses (abstract interpretation) with precise checks (SMT) in a staged pipeline.
 - For concurrent systems, model-check small instances to find concurrency bugs (then generalize proofs).
-

13. Summary

- **Verification** is the formal study of program correctness; several approaches exist (deduction, abstract interpretation, model checking, proof assistants).
 - **Hoare Logic** and WP let us derive preconditions and structure proofs.
 - **Loop invariants** and **variants** are central for reasoning about loops and termination.
 - **Abstract interpretation** trades precision for scalability; **model checking** trades scalability for exhaustiveness on finite-state models.
 - A pragmatic verification workflow mixes tools to get early feedback (fast over-approx), deep proofs for critical components, and model checking for protocols.
-